```python
from collections import deque

start = (3, 3, 'L')
goal = (0, 0, 'R')

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if m > 0 and m < c:  # missionaries outnumbered on left
        return False
    if (3 - m) > 0 and (3 - m) < (3 - c):  # missionaries outnumbered on right
        return False
    return True

def next_states(state):
    m, c, side = state
    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
    states = []

    for dm, dc in moves:
        if side == 'L':
            nm = m - dm
            nc = c - dc
            ns = 'R'
        else:
```

Run

```
            ns = 'R'
        else:
            nm = m + dm
            nc = c + dc
            ns = 'L'

        if is_valid(nm, nc):
            states.append(((nm, nc, ns
                ), (dm, dc)))  #
                include boat passengers
    return states

def bfs():
    q = deque([(start,[start],[] )])  #
        (state, path, moves)
    visited = set()

    while q:
        state, path, moves = q.popleft
            ()

        if state == goal:
            return path, moves

        if state in visited:
            continue

        visited.add(state)

        for s, move in next_states
```

Run

```
42              return path, moves
43
44          if state in visited:
45              continue
46
47          visited.add(state)
48
49          for s, move in next_states
                (state):
50              q.append((s, path+[s],
                    moves+[move]))
51
52  solution, boat_moves = bfs()
53
54  print("Solution:")
55  for i, step in enumerate(solution):
56      m, c, side = step
57      left = (m, c)
58      right = (3 - m, 3 - c)
59      if i == 0:
60          print(f"Step {i}: Left={left},
                Right={right}, Boat={side}"
                )
61      else:
62          dm, dc = boat_moves[i-1]
63          print(f"Step {i}: Left={left},
                Right={right}, Boat={side},
                "
64                f"Boat carried {d...}"
```

Run

```
Solution:
Step 0: Left=(3, 3), Right=(0, 0), Boat=L
Step 1: Left=(3, 1), Right=(0, 2), Boat=R,
    Boat carried 0M 2C
Step 2: Left=(3, 2), Right=(0, 1), Boat=L,
    Boat carried 0M 1C
Step 3: Left=(3, 0), Right=(0, 3), Boat=R,
    Boat carried 0M 2C
Step 4: Left=(3, 1), Right=(0, 2), Boat=L,
    Boat carried 0M 1C
Step 5: Left=(1, 1), Right=(2, 2), Boat=R,
    Boat carried 2M 0C
Step 6: Left=(2, 2), Right=(1, 1), Boat=L,
    Boat carried 1M 1C
Step 7: Left=(0, 2), Right=(3, 1), Boat=R,
    Boat carried 2M 0C
Step 8: Left=(0, 3), Right=(3, 0), Boat=L,
    Boat carried 0M 1C
Step 9: Left=(0, 1), Right=(3, 2), Boat=R,
    Boat carried 0M 2C
Step 10: Left=(1, 1), Right=(2, 2), Boat=L,
    Boat carried 1M 0C
Step 11: Left=(0, 0), Right=(3, 3), Boat=R,
    Boat carried 1M 1C

=== Code Execution Successful ===
```